

专业 计算机科学与技术 班级 计科 20215 班 成绩

学号 20211018341 姓名 卢瑶

实验四 ARM 汇编语言程序设计实验(一)

一、 实验目的

1. 掌握 ADS1.2 集成开发环境
2. 了解 ARM 汇编指令用法，并能够编写简单的汇编程序
3. 掌握指令的条件执行，掌握 LDR/STR 指令，完成存储器的

访问

二、 实验环境

硬件：PC 机 Pentium100 以上。

软件：PC 机 Windows 操作系统、ARM ASD 1.2 集成开发环境、

AXD

三、 实验内容

具体内容：

1. 用 LDR 指令读取 0x40003100 地址上的数据，将该数据加 1，若结果大于 10，则使用 STR 指令将结果写入原地址，否则，将把 0 写入原地址。

2. 用 ADS1.2 软件仿真，单步、全速运行程序，设置断点，打开寄存器窗口（ProcessorRegister）监视 R0、R1 的值，打开存储器观察窗口（Memory）监视 0x40003100 地址处的值。

实验步骤：

1. 启动 ADS1.2, 使用 ARM Executable Image 工程模板建立一个工程。如 SY4

2. 建立汇编源文件 Test4.s, 加入工程中。

3. 设置工程连接地址 R0 Base 0x40000000, RW Base 0x40003000。

4. 编译、连接工程, 选择 Project□Debug, 启动 AXD 软件仿真调试。

5. 打开寄存器窗口, 监视 R0、R1 的值, 设置观察地址 0x40003100, 显示方式为 32bit, 监测 0x40003100 上的值。

6. 可以单步运行程序, 可以设置、取消断点, 或者全速运行, 停止运行, 调试时观察寄存器 0x40003100 上的值。

结果如下图所示, 实现了用 LDR 指令读取 0x40003100 地址上的数据, 将该数据加 1, 若结果大于 10, 则置零, 否则将该数存入存储器中。MOVHS 用法则是若 R0 大于 10 (CMP 语句), 则将 R0 赋值为 0。

```
1. COUNT EQU 0X40003100
2. AREA TEST3, CODE, READONLY
3. ENTRY
4. CODE32
5. START
6. LDR R1, =COUNT
7. MOV R0, #0
8. STR R0, [R1]
9. LOOP
10. LDR R1, =COUNT
11. LDR R0, [R1]
12. ADD R0, R0, #1
13. CMP R0, #10
14. MOVHS R0, #0 ; 如果>=10 那么执行这条语句
```

15. STR R0,[R1]

16. B LOOP

17. END

成功编译到 AXD Debugger 执行：

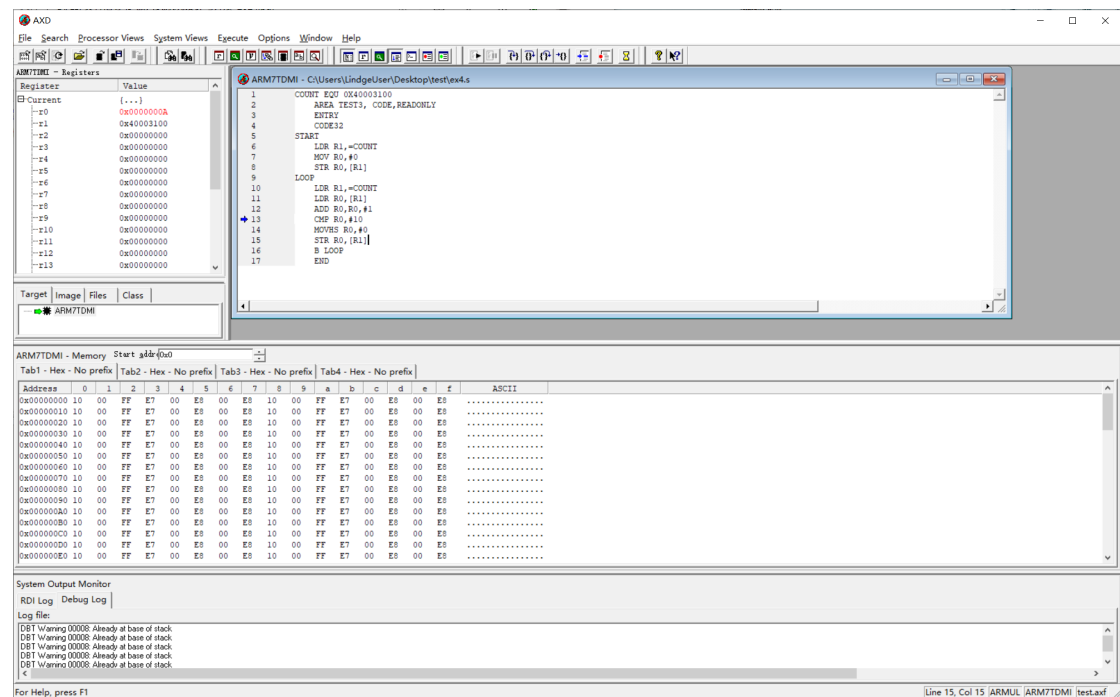


图 1 运行过程图-1

指令执行到 STR R0,[R1] 寄存器变化如下所示：

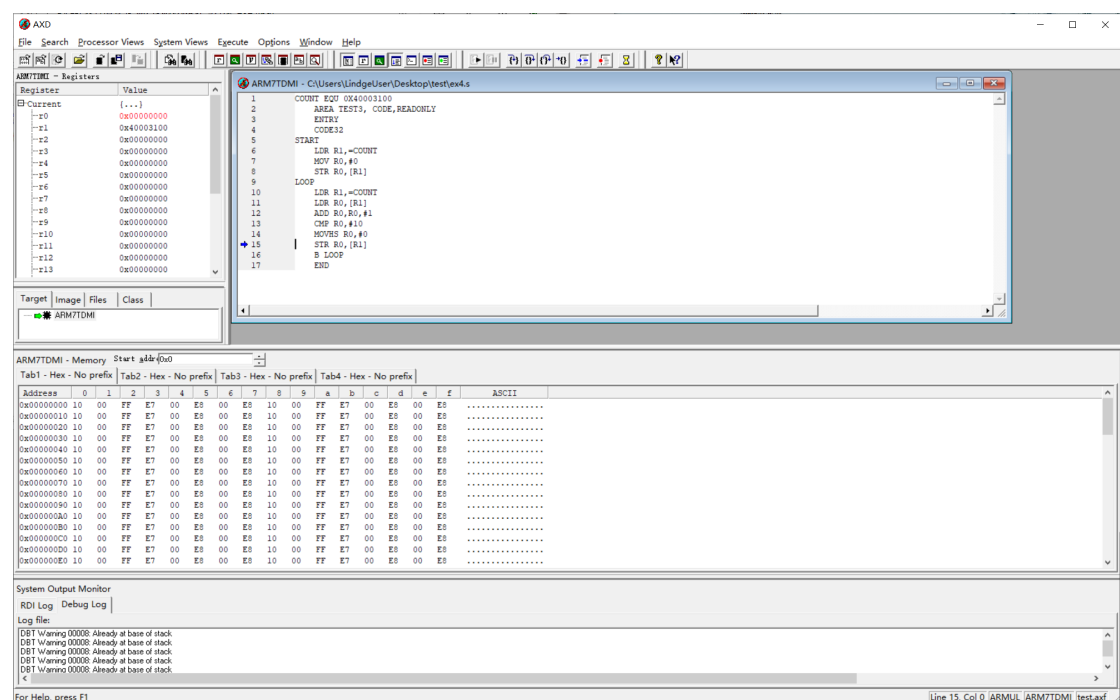


图 2 执行过程图-2

指令执行完毕，寄存器的最终变化如下图所示：

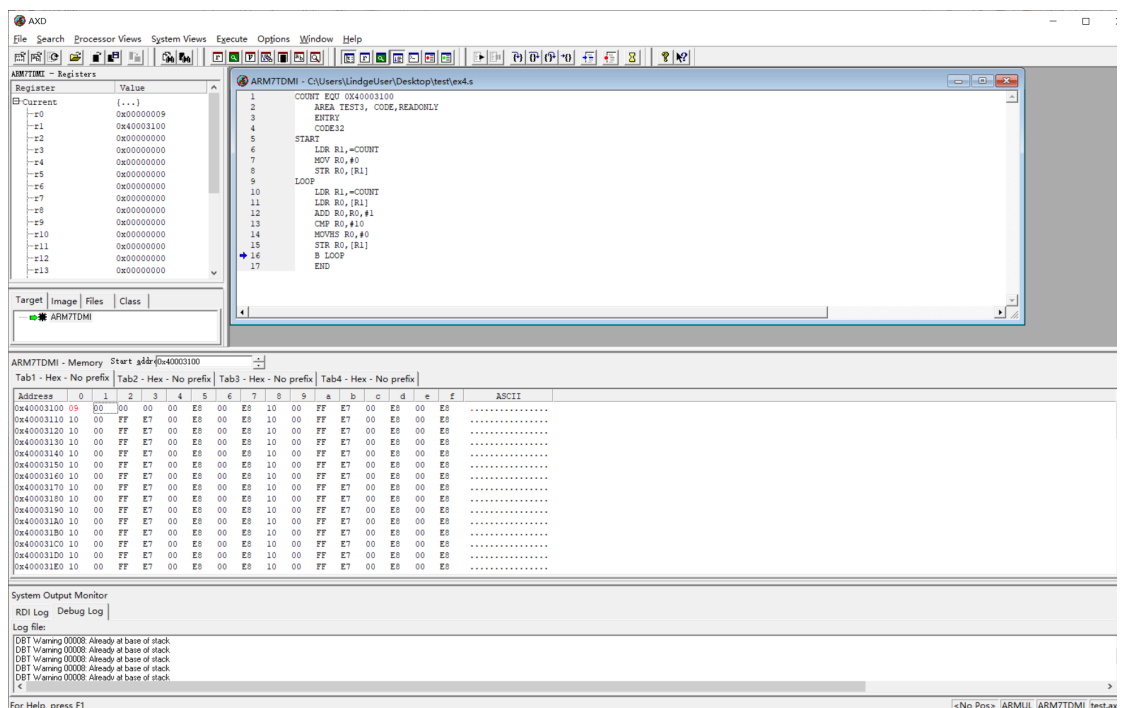


图 3 最终的寄存器状态图

四、实验心得

1. 收获和体会

在此次 ARM 汇编语言程序设计实验中，我通过编写代码和使用 ADS1.2 软件仿真，进一步学习了 LDR 和 STR 指令的使用方法，并通过设置断点、监视寄存器和存储器的方式来调试和验证程序的正确性。

以下是我的主要收获：

- **数据加载与存储：**使用 LDR 指令从地址 0x40003100 读取数据并存储在寄存器 R0 中，然后通过 ADD 指令将 R0 的值加 1。这个过程让我深入理解了数据加载与存储的基本操作。
- **条件执行：**使用 CMP 指令比较寄存器 R0 的值与 10 的大小，并根据结果使用 MOVHS 指令实现条件操作。如果 R0 的值大于或等于 10，则将 R0 的值置为 0；否则，保持 R0 的值不变。

这让我理解了 ARM 汇编语言中的条件执行机制。

- **调试与验证：**在仿真过程中，我利用 ADS1.2 软件进行单步调试和全速运行，设置断点以逐步分析和验证代码。通过寄存器窗口实时监视 R0 和 R1 的值变化，以及存储器观察窗口监视 0x40003100 地址处的值，确保了程序的正确性和功能实现。

2. 遇到的问题及解决方法

在本次实验中，我没有遇到明显的问题。代码的逻辑相对简单，使用了常量和寄存器的正确操作，以及合适的条件判断和数据写入。以下是一些具体的观察和思考：

- **正确使用 LDR 和 STR 指令：**在实验过程中，我意识到正确使用 LDR 和 STR 指令对于访问存储器至关重要。确保地址和数据的准确性是保证程序正确运行的关键。

- **条件执行的细节：**条件执行是 ARM 汇编语言的一大特色。在实验中，我通过 CMP 指令比较寄存器值，并根据结果使用 MOVHS 指令进行条件操作，体会到了这种条件执行机制的灵活性和高效性。

- **调试的重要性：**通过设置断点和单步调试，我能够逐步分析程序的执行情况，及时发现和纠正潜在的问题。这对于理解程序的执行流程和确保其正确性非常重要。

通过这次实验，我不仅熟悉了 ARM 汇编语言的基本指令和操作，还学会了如何在仿真环境中调试和验证程序。这些技能将对我今后的

学习和工作大有裨益。